

Hearing the Platonic Solids

Graph Sonification

Darshani P. Gole

In spectral graph sonification, different components of the spectral decomposition encode different aspects of graph structure. In this work, eigenvalues and eigenspaces are used to generate distinct auditory representations of the graph.

Eigenvalue-Based Sonification

Eigenvalues of the Laplacian matrix of the graph are mapped to audible frequencies. Each eigenvalue represents a spectral mode of the graph, and the distribution of eigenvalues determines the overall spectral character of the resulting sound.

Generation of the Eigenvalue Sonification

Given a graph G , its adjacency matrix A is used to construct the graph Laplacian

$$L = D - A$$

where D is the degree matrix. The eigenvalues of L are then computed:

$$0 = \lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_n$$

Distinct eigenvalues are mapped to audible frequencies. Larger eigenvalues are assigned higher pitches, while smaller eigenvalues are assigned lower pitches.

To represent eigenvalue multiplicity, multiple slightly detuned copies of the corresponding frequency are generated. The interference between these nearby frequencies produces an audible beating effect. Consequently:

- pitch represents the eigenvalue,
- beating or wobbling represents the multiplicity of the eigenvalue.

Since high multiplicities often arise from graph symmetries, strongly symmetric graphs typically produce more pronounced beating effects.

All generated tones are summed and normalized to produce the final audio signal, which is exported as a WAV file.

Platonic Solids

- Tetrahedron
- Cube
- Octahedron
- Dodecahedron
- Icosahedron

Archimedean Solids

- Truncated Tetrahedron
- Truncated Cube
- Truncated Octahedron
- Truncated Dodecahedron
- Truncated Icosahedron
- Cuboctahedron
- Icosidodecahedron
- Rhombicuboctahedron
- Rhombicosidodecahedron
- Great rhombicuboctahedron
- Great rhombicosadodecahedron
- Snub cube
- Snub dodecahedron

Other graphs

- Random tree with 20 vertices
- Star graph with 10 vertices
- Path on 5 vertices
- Cycle C_5
- Complete graph K_7
- Complete Bipartite graph $K_{3,5}$

Eigenspace-Based Sonification

In the eigenspace sonification, each eigenspace is represented by a pitch associated with its eigenvalue. The dimension of the eigenspace (equivalently, the multiplicity of the corresponding eigenvalue) is encoded through the richness of the sound.

Unlike the eigenvalue sonification, which emphasizes the spectrum itself, the eigenspace sonification emphasizes the dimensions of the corresponding eigenspaces.

Generation of the Eigenspace Sonification

The graph Laplacian

$$L = D - A$$

is constructed and its spectral decomposition is computed. For each distinct eigenvalue λ , the corresponding eigenspace

$$E_\lambda = \{v \in \mathbb{R}^n : Lv = \lambda v\}$$

is identified.

Each eigenspace is assigned a base frequency determined by its eigenvalue. The dimension of the eigenspace, equal to the multiplicity of the eigenvalue, is encoded through harmonic enrichment.

For an eigenspace of dimension m , the generated sound consists of the fundamental frequency together with its first m harmonics:

$$\sin(2\pi ft) + \frac{1}{2} \sin(2\pi 2ft) + \dots + \frac{1}{m} \sin(2\pi mft)$$

Thus:

- pitch represents the eigenvalue associated with the eigenspace,
- timbre or harmonic richness represents the dimension of the eigenspace.

The sounds corresponding to all eigenspaces are combined and normalized to obtain the final audio signal, which is exported as a WAV file.

Platonic Solids

- Tetrahedron
- Cube
- Octahedron

- Dodecahedron
- Icosahedron

Archimedean Solids

- Truncated Tetrahedron
- Truncated Cube
- Truncated Octahedron
- Truncated Dodecahedron
- Truncated Icosahedron
- Cuboctahedron
- Icosidodecahedron
- Rhombicuboctahedron
- Rhombicosidodecahedron
- Great rhombicuboctahedron
- Great rhombicosadodecahedron
- Snub cube
- Snub dodecahedron

Other graphs

- Random tree with 20 vertices
- Star graph with 10 vertices
- Path on 5 vertices
- Cycle C_5
- Complete graph K_7
- Complete Bipartite graph $K_{3,5}$

General code used to generate audio

```
import numpy as np
from scipy.linalg import eigh
from scipy.io.wavfile import write

class SpectralSonifier:
    """
```

Spectral graph sonification based on Laplacian eigenvalues and eigenspace multiplicities.

"""

```
def __init__(self, A, name="graph", sr=44100):
```

```
    self.A = np.array(A, dtype=float)
```

```
    self.name = name
```

```
    self.sr = sr
```

```
    # Laplacian
```

```
    D = np.diag(np.sum(self.A, axis=1))
```

```
    self.L = D - self.A
```

```
    # Spectral decomposition
```

```
    self.evals, self.evecs = eigh(self.L)
```

```
# -----
```

```
# Utility
```

```
# -----
```

```
def save_wav(self, signal, filename):
```

```
    signal = signal / (np.max(np.abs(signal)) + 1e-12)
```

```
    audio = (32767 * signal).astype(np.int16)
```

```
    write(filename, self.sr, audio)
```

```
    print(f"Saved: {filename}")
```

```
# -----
```

```
# Group equal eigenvalues
```

```
# -----
```

```
def spectral_blocks(self, tol=1e-6):
```

```
    blocks = []
```

```
    i = 0
```

```
    n = len(self.evals)
```

```
    while i < n:
```

```
        lam = self.evals[i]
```

```

        j = i + 1

        while j < n and abs(self.evals[j] - lam) < tol:
            j += 1

        multiplicity = j - i

        blocks.append((lam, multiplicity))

        i = j

    return blocks

# -----
# Print spectrum
# -----

def print_spectrum(self):

    print("\nSpectrum:\n")

    for lam, mult in self.spectral_blocks():
        print(f" = {lam:.4f}, multiplicity = {mult}")

# -----
# 1. Eigenvalue sonification
# -----

def eigenvalue_sonify(self, duration=5, base_freq=220):

    t = np.linspace(0, duration, int(self.sr * duration), endpoint=False)

    signal = np.zeros_like(t)

    blocks = self.spectral_blocks()

    max_lambda = max(lam for lam, _ in blocks)

    for lam, mult in blocks:

        freq = base_freq * 2 ** (lam / (max_lambda + 1e-9))

        # multiplicity -> chord size
        for k in range(mult):

```

```

        detune = 1 + 0.01 * (k - (mult - 1) / 2)

        signal += (1 / mult) * np.sin(2 * np.pi * freq * detune * t)

    self.save_wav(signal, f"{self.name}_eigenvalues.wav")

# -----
# 2. Eigenspace sonification
# -----

def eigenspace_sonify(self, duration=5, base_freq=220):

    t = np.linspace(0, duration, int(self.sr * duration), endpoint=False)

    signal = np.zeros_like(t)

    blocks = self.spectral_blocks()

    max_lambda = max(lam for lam, _ in blocks)

    for lam, mult in blocks:

        freq = base_freq * 2 ** (lam / (max_lambda + 1e-9))

        # multiplicity controls richness
        for h in range(1, mult + 1):

            signal += (1 / h) * np.sin(2 * np.pi * h * freq * t)

    self.save_wav(signal, f"{self.name}_eigenspaces.wav")

# -----
# 3. Combined sonification
# -----

def combined_sonify(self, duration=6, base_freq=220):

    t = np.linspace(0, duration, int(self.sr * duration), endpoint=False)

    signal = np.zeros_like(t)

    blocks = self.spectral_blocks()

    max_lambda = max(lam for lam, _ in blocks)

```

```

    for lam, mult in blocks:

        freq = base_freq * 2 ** (lam / (max_lambda + 1e-9))

        # main spectral tone
        signal += np.sin(2 * np.pi * freq * t)

        # multiplicity texture
        for h in range(2, mult + 2):

            signal += (0.5 / h) * np.sin(2 * np.pi * h * freq * t)

        self.save_wav(signal, f"{self.name}_combined.wav")

# -----
# Generate everything
# -----

def generate_all(self):

    self.print_spectrum()

    self.eigenvalue_sonify()

    self.eigenspace_sonify()

    self.combined_sonify()

    print(f"\nFinished generating audio for {self.name}")

```

The code also generates an extra combined.wav file. The combined sonification is obtained by superposing the sounds associated with all eigenspaces. The resulting audio provides a global auditory representation of the spectral decomposition, simultaneously reflecting the distribution of eigenvalues and the dimensions of the corresponding eigenspaces.

For simplicity purpose, the combined versions are not included here.